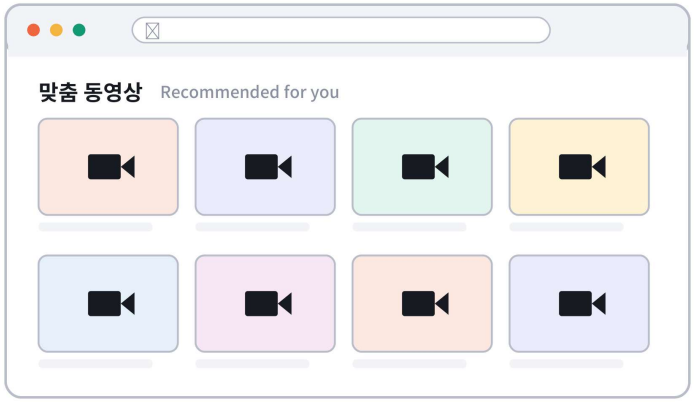
인공지능플랫폼 실습/설계 · 9차시

동영상 추천 시스템

Video Recommendation System



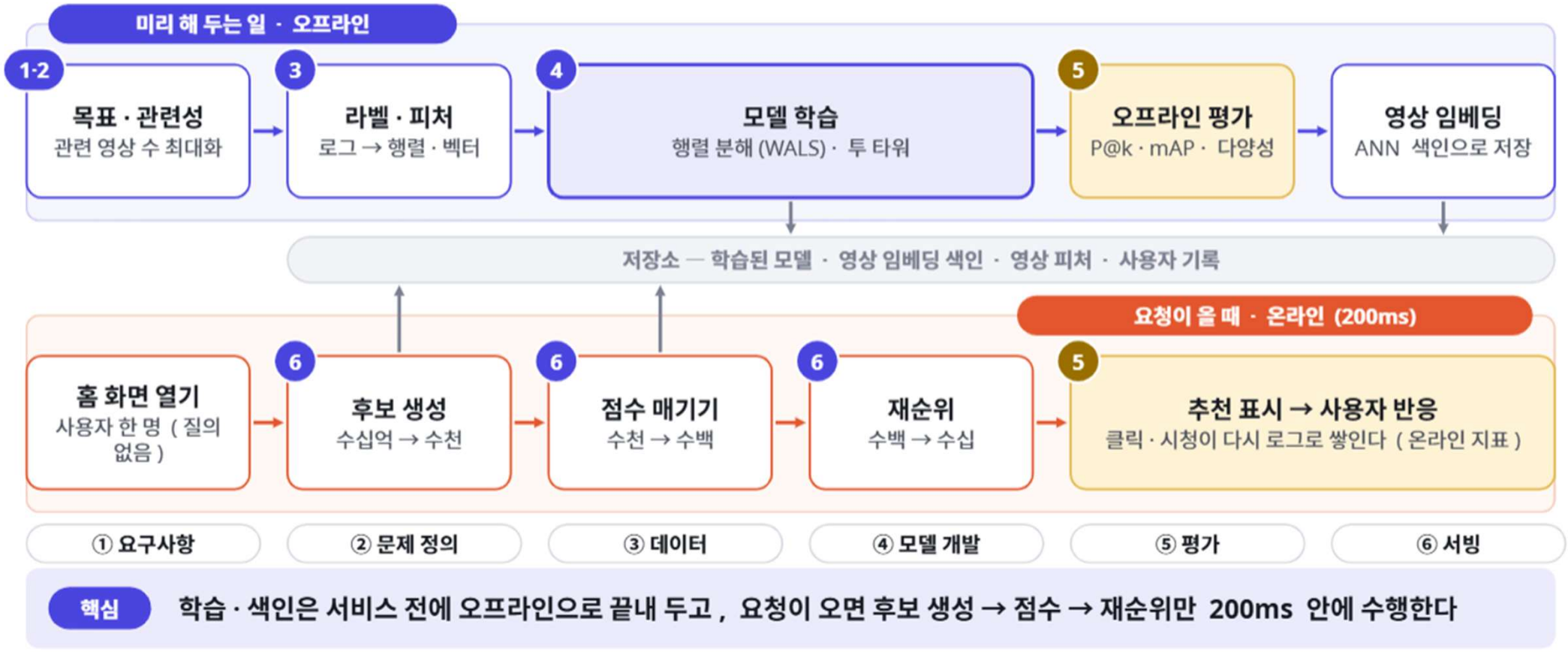
홈 화면을 여는 순간, 무엇을 보여 줄 것인가?

오늘의 여정 — 시스템 설계 6단계

1 요구사항 — 2 문제 정의 — 3 데이터 — 4 모델 개발 — 5 평가 — 6 서빙

덕성여자대학교 디지털소프트웨어공학부 · 원문: ByteByteGo, Machine Learning System Design Interview, Ch.6

시스템 전체 구조 — 오프라인 학습과 온라인 서빙



학습 로드맵 — 시스템 설계 6단계의 핵심 개념

① 요구사항	기능·비기능 요구사항 (규모, 지연 시간), 설계 범위
② 문제 정의	ML 목표 · 관련성 정의 · 콘텐츠 기반 / 협업 필터링 / 하이브리드
③ 데이터	상호작용 로그 · 라벨 · 피드백 행렬 · 피처 엔지니어링
④ 모델 개발	행렬 분해(손실 함수 · WALS) · 투 타워 신경망 · 모델 비교
⑤ 평가	오프라인 지표 (P@k · mAP · 다양성) · 온라인 지표 (CTR 등)
⑥ 서빙	후보 생성 → 점수 매기기 → 재순위 · 콜드 스타트 · 과제

핵심 **요구사항 → 문제 정의 → 데이터 → 모델 개발 → 평가 → 서빙**

요구사항 분석 — 규모(100억 개)와 응답 시간(200ms)

100억

동영상 수 (전 세계·다국어)

↓

전부 점수 매길 수 없다
→ 후보를 먼저 줄이는 다단계 구조

200ms

추천 응답 시간

↓

무거운 모델을 통째로 못 쓴다
→ 무거운 모델은 줄어든 후보에만

목표	사용자 참여 ↑
형태	홈 화면 개인화 (질의 없음)
범위	전 세계 · 다국어 → 언어 피처
제외	재생목록 (단순화)

핵심 **동영상 100억 개를 200ms 안에 추천해야 하므로 전체를 한 번에 점수 매길 수 없고, 후보를 줄이는 다단계 구조가 필요하다**

ML 목표 정의 — 참여도를 무엇으로 측정할 것인가

1 클릭 수 최대화

× 낚시 (clickbait) 영상이 올라간다

제목 · 썸네일만 그럴듯 , 실제는 지루하거나 오해를 부른다 → 장기적으로 만족도 ↓

2 완주 영상 수 최대화

× 짧은 영상만 추천한다

빨리 끝나는 영상이 유리해진다

3 총 시청 시간 최대화

× 오래 보는 영상을 추천한다

원문은 별도 단점을 언급하지 않는다

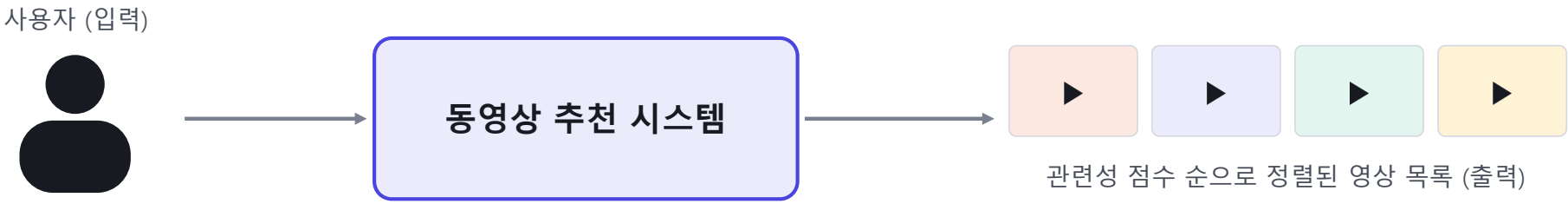
4 관련 영상 수 최대화

✓ 관련성을 규칙으로 직접 정의

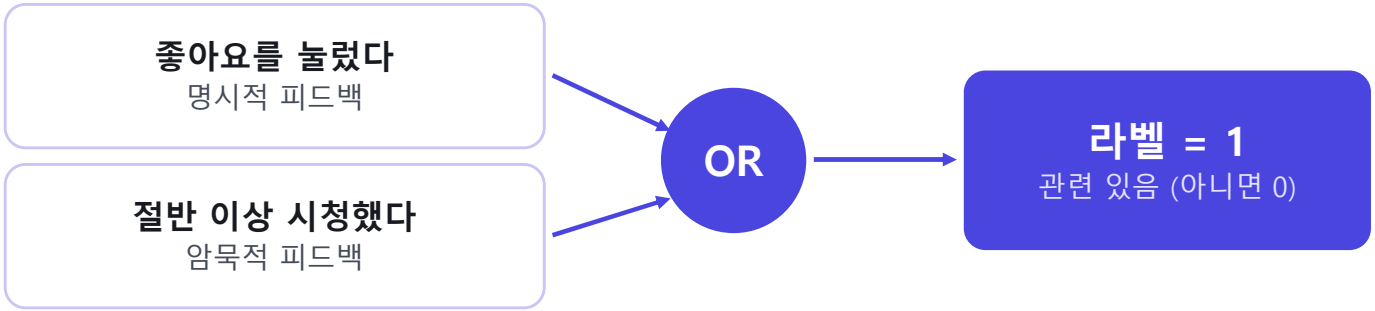
어떤 신호 (좋아요 · 시청 비율) 를 쓸지 우리가 통제 → 위 부작용이 없다

핵심 클릭 · 완주 · 시청 시간 최대화는 부작용이 있어 , 관련성을 규칙으로 정의하고 " 관련 영상 수 " 최대화를 ML 목표로 정한다

입력·출력과 관련성 정의 — 라벨 규칙



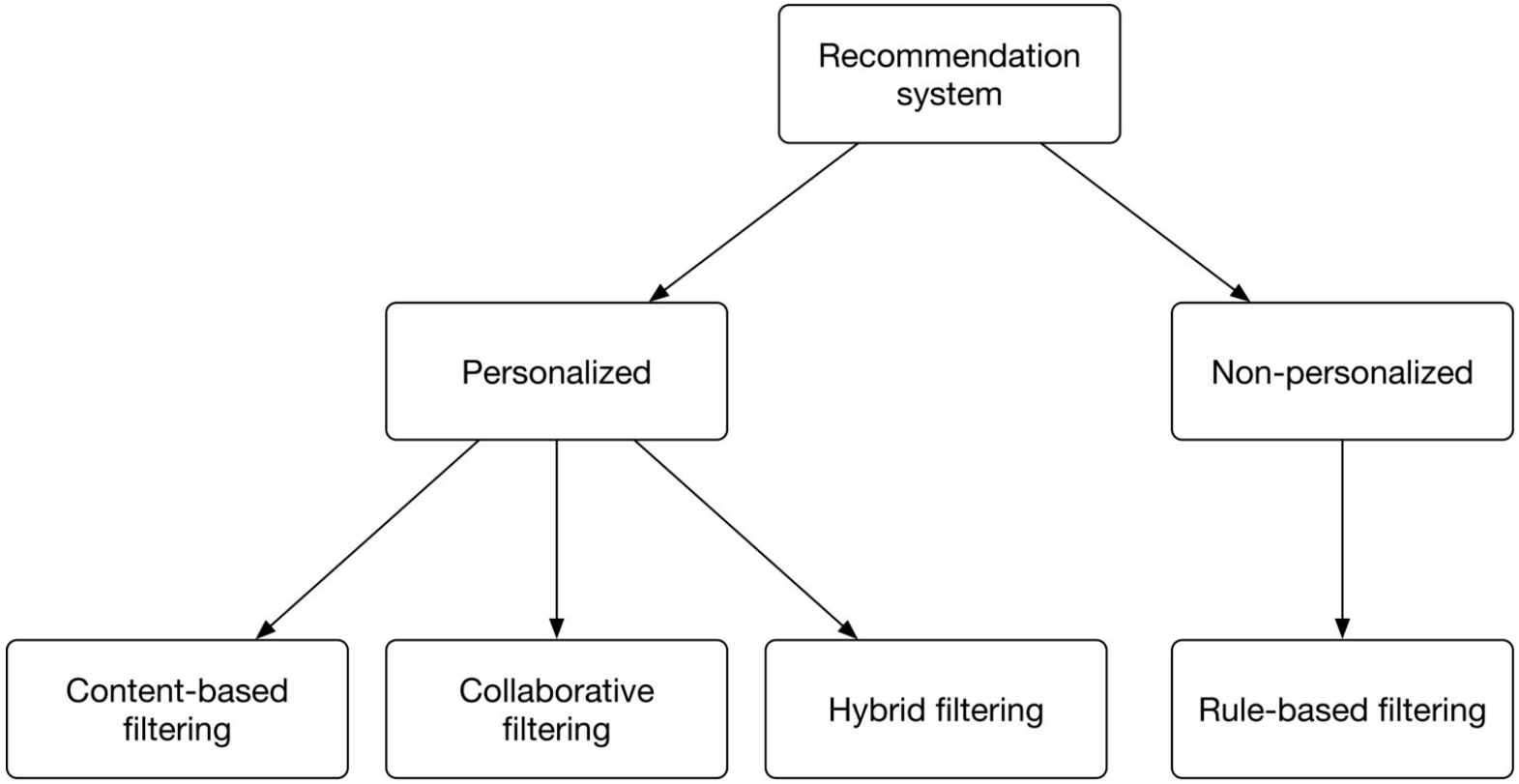
관련성 정의 규칙 → 학습 데이터의 정답(라벨)



실습에서 확인
좋아요만 쓰면 양성 7.9%
결합하면 26.1%
→ 명시적 피드백은 희소하다

핵심 **관련성의 정의: 좋아요를 눌렀거나 영상을 절반 이상 시청했으면 관련 있음(1), 아니면 0 — 학습 라벨로 활용**

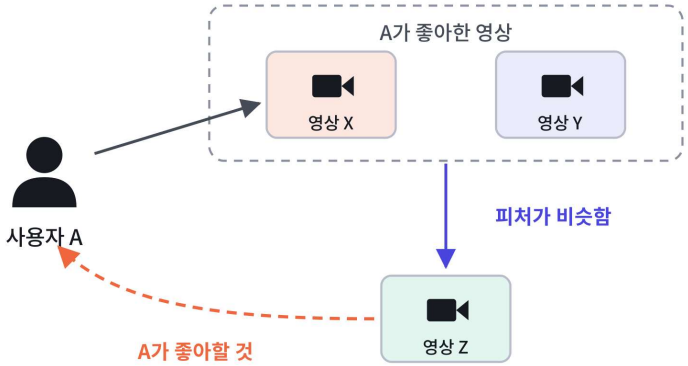
일반적인 추천 시스템 유형



추천 방식의 분류 — 콘텐츠 기반 필터링과 협업 필터링

콘텐츠 기반 필터링

영상의 피처가 비슷하면 추천

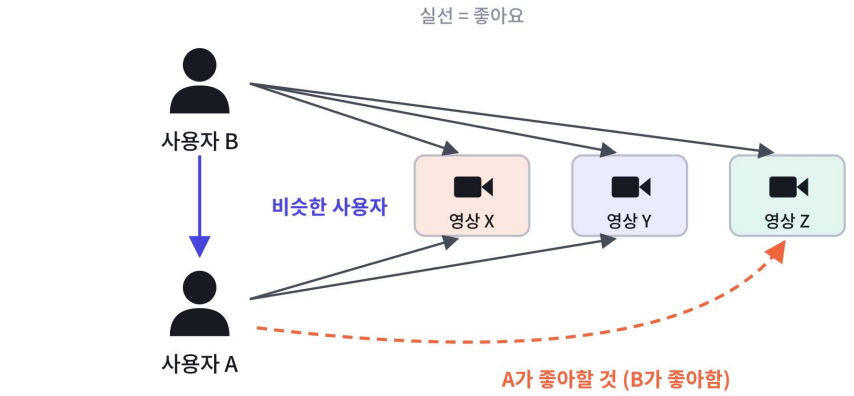


✓ 새 영상도 추천 가능. 사용자의 고유한 관심사 파악 가능

× 새 관심사를 발견하기 어렵다, 동영상 피처를 엔지니어링 하야 하는 경우, 도메인 지식이 필요하다.

협업 필터링 (CF)

나와 비슷한 사람이 본 영상을 추천



✓ 새 관심 분야를 쉽게 발견할 수 있다.
동영상 피처를 사용하지 않으므로 동영상에서 피처를 찾기 위한 도메인 지식이 필요하지 않다.
같은 이유로 콘텐츠기반 필터링 보다 빠르다.

× 콜드스타트 문제 : 새 영상·새 사용자에게 적용할 수 없다.
(상호작용 데이터의 부족) 특이한 관심사에 대응하기 어렵다.
(유사 사용자를 찾기 어려움)

핵심 **콘텐츠 기반은 영상 피처의 유사도로, 협업 필터링은 사용자 행동의 유사도로 영상을 추천한다**

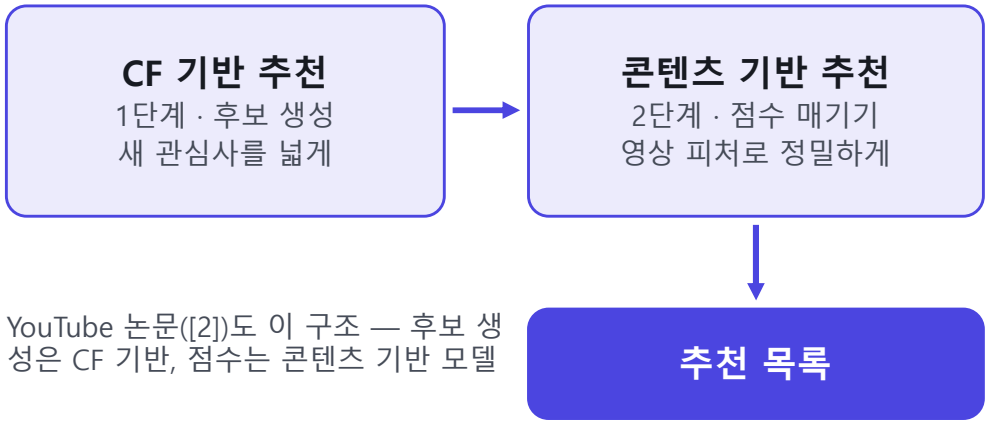
하이브리드 추천 — 두 방식의 비교와 순차 결합

표 6.1

	콘텐츠 기반	협업 필터링
새 영상 처리	✓	×
새 관심 분야 발견	×	✓
도메인 지식 불필요	×	✓
효율성	×	✓

← 앞 두 줄은 정확히 반대

순차 하이브리드 (실무에서 주로 사용)



YouTube 논문([2])도 이 구조 — 후보 생성은 CF 기반, 점수는 콘텐츠 기반 모델

병렬 방식: 두 결과를 결합기로 합친다 (원문 그림 6.6, 실무는 순차를 선호)

핵심 **두 방식은 약점이 서로 반대이므로, 협업 필터링으로 후보를 넓힌 뒤 콘텐츠 기반으로 점수를 매긴다 (순차 하이브리드)**

하이브리드 필터링 — 병렬 또는 순차

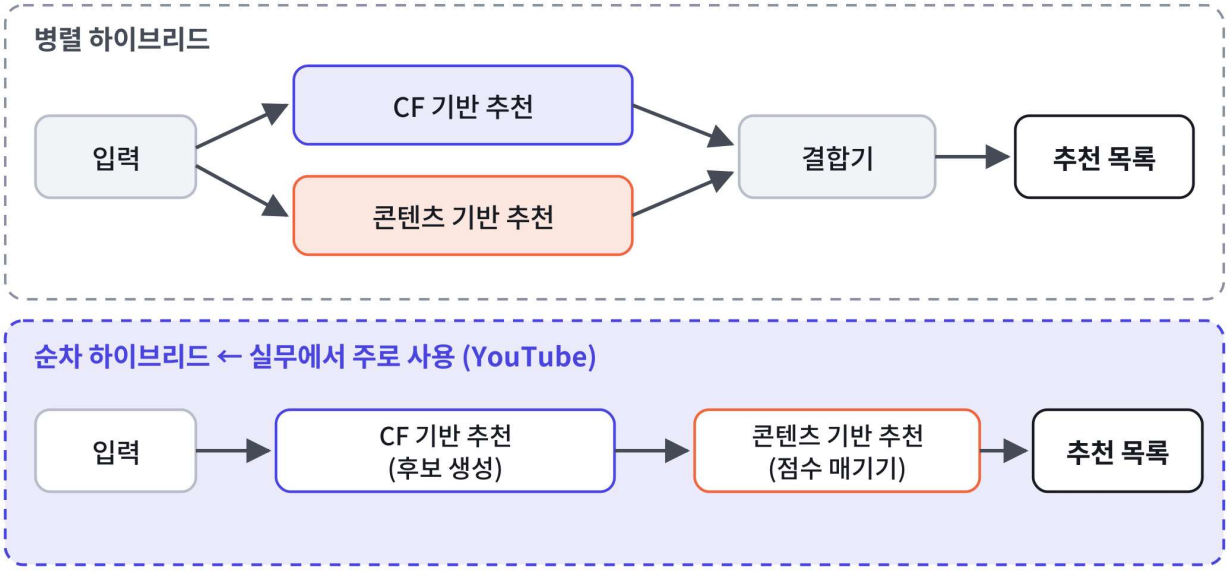


그림 6.6

왜 더 좋은가

두 데이터 원천을 쓴다 — 과거 상호작용(CF)과 영상 피처(CB). CF는 새 관심사, CB는 정밀한 관련성.

우리의 선택 ✓

YouTube 논문처럼 순차 하이브리드: 1단계 CF 기반 후보 생성 → 2단계 콘텐츠 기반 점수 매기기.

추천 방식 구현 — 콘텐츠 기반과 협업 필터링 코드

영상 0 스키 · 1 스노보드 · 2 파스타 · 3 김치찌개 · 4 겨울 캠핑 · 5 파이썬 사용자 A가 본 영상: 0, 1

콘텐츠 기반 — 태그로 유사도

```
vec = CountVectorizer(binary=True, token_pattern=r'\S+')
X_toy = vec.fit_transform(toy.tags).toarray()  # 태그→0/1

liked = [0, 1]                                 # A가 본 영상
profile = X_toy[liked].mean(axis=0)            # ① 프로필=평균
scores = cosine_similarity([profile], X_toy)[0]  # ② 유사도
scores[liked] = -1                             # ③ 본 영상 제외
```

협업 필터링 — 행동으로 유사도

```
R = np.array([[1,1,0,0,0,0],   # A
              [1,1,0,0,0,1],   # B
              [0,0,1,1,0,0],   # C
              [0,0,1,0,1,0]])  # D
sim_users = cosine_similarity(R)   # 사용자×사용자
np.fill_diagonal(sim_users, 0)
scores = sim_users[0] @ R          # ① 비슷한 사람들의 선택
scores[R[0] == 1] = -1             # ③ 본 영상 제외
```

추천 1위 → 겨울 캠핑 브이로그 (0.365)
이유: 태그 '겨울'을 공유 — 안 겹치면 0점

추천 1위 → 파이썬 기초 (0.816)
이유: 비슷한 사용자 B가 봤다 — 태그는 하나도 안 겹침

핵심 **두 방식 모두 "유사도 점수 → 본 영상 제외 → 최고점 추천" 순서이고, 유사도를 재는 기준(태그/행동)만 다르다**

두 방식의 한계 확인 — 새 영상과 새 관심사 (콜드 스타트)

	콘텐츠 기반	협업 필터링
새 영상 '눈썰매 타기' — 방금 업로드, 아무도 안 봄	✓ 점수 0.632 바로 추천 가능 — 태그만 있으면 점수가 계산된다	× 점수 0.000 열이 전부 0 → 영원히 0점 (콜드 스타트)
새 관심사 '파이썬 기초' — A의 태그와 하나도 안 겹침	× 점수 0.000 태그가 안 겹치면 0점 — 새 관심사를 못 찾는다	✓ 점수 0.816 비슷한 사용자 B가 봤다 → 1위

핵심 **새 영상은 협업 필터링이 추천할 수 없고(점수 0), 태그가 안 겹치는 영상은 콘텐츠 기반이 추천할 수 없다(점수 0)**

구현 예제 — 한 셀로 실행하는 콘텐츠 기반 · 협업 필터링

```
import numpy as np
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics.pairwise import cosine_similarity

titles = ['스키 강습', '스노보드 묘기', '파스타',
          '김치찌개', '겨울 캠핑', '파이썬 기초']
tags   = ['스키 겨울 운동', '스노보드 겨울 운동', '요리 이탈리아',
          '요리 한식', '캠핑 겨울 여행', '코딩 파이썬']

# ① 콘텐츠 기반: A가 좋아한 0, 1번과 태그가 비슷한 영상
vec = CountVectorizer(binary=True, token_pattern=r'\S+')
X = vec.fit_transform(tags).toarray()
s = cosine_similarity([X[[0, 1]].mean(axis=0)], X)[0]
s[[0, 1]] = -1
print('콘텐츠 기반 →', titles[s.argmax()])

# ② 협업 필터링: A와 비슷한 사용자가 본 영상
R = np.array([[1, 1, 0, 0, 0, 0],   # A
              [1, 1, 0, 0, 0, 1],   # B
              [0, 0, 1, 1, 0, 0],   # C
              [0, 0, 1, 0, 1, 0]])  # D
sim = cosine_similarity(R); np.fill_diagonal(sim, 0)
s = sim[0] @ R
s[R[0] == 1] = -1
print('협업 필터링 →', titles[s.argmax()])
```

실행 결과

콘텐츠 기반 → 겨울 캠핑
협업 필터링 → 파이썬 기초

바꿔 보기

A가 본 영상을 [2, 3]으로
→ 콘텐츠 기반은 무엇을 찾을까?

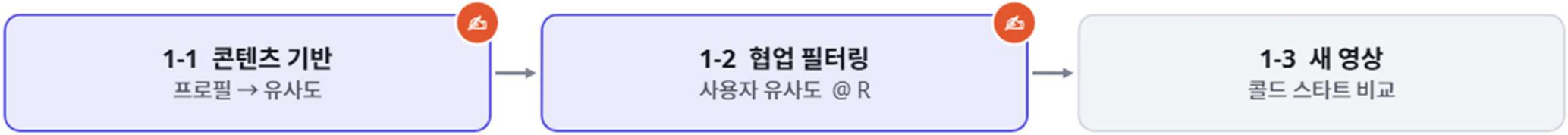
D 행에 파이썬(1) 추가
→ 협업 필터링 점수는?

태그에 "겨울"을 하나 더 넣기

실습

실습 1 · 콘텐츠 기반과 협업 필터링 구현 (빈칸 ①②)

HANDS-ON 01 약 15 분 · 9 차시 _ 실습 _ 동영상추천시스템 .ipynb · 1 절



✍ 빈칸 ①

```
profile = ____    # 좋아한 영상 피처의 평균
```

✍ 빈칸 ②

```
scores = ____    # 비슷한 사용자들의 선택 가중합
```

통과 신호

셀을 실행해 ☒ 통과가 출력되면 성공입니다 .

막히면 셀 위 접힌 " 힌트 " 를 펼치세요 .

관찰 : 파이썬 기초가 1 위인 이유를 태그와 사용자 관점에서 한 문장으로 설명해 보세요 .

세 가지 데이터 — 영상 · 사용자 · 상호작용

영상 메타데이터 (표 6.2)

ID	길이(초)	수동 태그	수동 제목	좋아요	조회수	언어
1	28	Dog, Family	Our lovely dog playing!	138	5300	English
2	300	Car, Oil	How to change your car oil?	5	250	Spanish
3	3600	Ouli, Vlog	Honeymoon to Bali	2200	255K	Arabic

사용자 (표 6.3)

ID	사용자명	나이	성별	도시	국가	언어	시간대

사용자-영상 상호작용 (표 6.4, 일부)

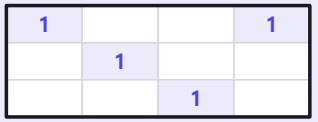
사용자	영상	유형	값	위치 (위도, 경도)	타임스탬프
4	18	좋아요	-	38.8951, -77.0364	1658451361
2	6	시청	46분	41.9241, -89.0389	1658822820
9	-	검색	Basics of clustering	22.7531, 47.9642	1659259402
8	6	댓글	Amazing video. Thanks	37.5189, 122.6405	1659244197

데이터 준비 — 원천 데이터에서 모델 입력까지

상호작용 로그
좋아요 · 시청 · 클릭 · 검색 · 댓글
+ 위치 · 시각

라벨 규칙
좋아요 OR 시청 ≥ 50%
→ 1

피드백 행렬



행렬 분해
상호작용만

기록 요약

사용자
나이 · 성별 · 언어 · 도시

동영상 메타데이터
길이 · 언어 · 제목 · 태그

피처화
범주형 → 임베딩
텍스트 → 벡터
수치 → 구간화
기록 → 평균

피처 벡터
사용자 피처
영상 피처

투 타워
피처 사용
새 사용자 OK

핵심 상호작용 로그는 피드백 행렬이 되어 행렬 분해의 입력이 되고, 사용자·영상 피처는 벡터가 되어 투 타워의 입력이 된다

상호작용 로그의 라벨 생성과 피드백 행렬

① 상호작용 로그 (표 6.4 단순화)

user	video	watch	like	label
0	6	0.09	0	**0**
0	38	0.00	0	**0**
0	90	0.79	1	**1**
0	51	0.55	0	**1**
…	…	…	…	**…**

② 라벨 규칙

```
log['label'] = (
  (log.like == 1)                # 명시적
  | (log.watch_frac >= .5)       # 암묵적
).astype(int)
```

③ 피드백 행렬 (그림 6.11)



1 = 관련 있음 · 빈칸 = 아직 모름

좋아요만 쓰면 양성 7.9% → 결합하면 26.1%
명시적 피드백만으로는 행렬이 너무 희소해 학습이 어렵다

빈칸 84.4%는 "싫다"가 아니라 "모른다"
빈칸의 의미가 다음 단계(손실 함수)의 핵심 질문이 된다

핵심 **라벨이 1인 (사용자, 영상) 칸만 모아 피드백 행렬을 만든다 — 빈칸은 "싫어함"이 아니라 "아직 모름"이다**

피처 엔지니어링 — 데이터 유형별 처리 방법

	원래 데이터	처리	벡터 (예시)
범주형	영상 ID · 언어 · 사용자 ID · 도시·국가	임베딩 층(학습) · 값이 적으면 원-핫	[0.1 0.9 0.2]
텍스트	제목 · 태그 · 검색어	제목 → BERT, 태그 → CBOW (실습: 태그 0/1 벡터)	[0.1 0.2 1.0]
수치	영상 길이 · 나이	정규화 / 나이는 구간화 + 원-핫	[0 1 0 0]
가변 길이 목록	좋아요·시청·노출 영상, 검색 기록	항목마다 임베딩 → 평균	[0.3 0.8 0.0]

실습은 사전학습 모델 없이 — 태그 0/1 + 언어 원-핫 + 길이/60 + 나이 구간 + 본 영상 태그 평균

핵심 **피처 처리 방법은 데이터 유형으로 정한다: 범주형은 임베딩·원-핫, 텍스트는 벡터화, 수치는 정규화·구간화, 목록은 평균**

동영상 피처 — 무엇을 어떻게 벡터로

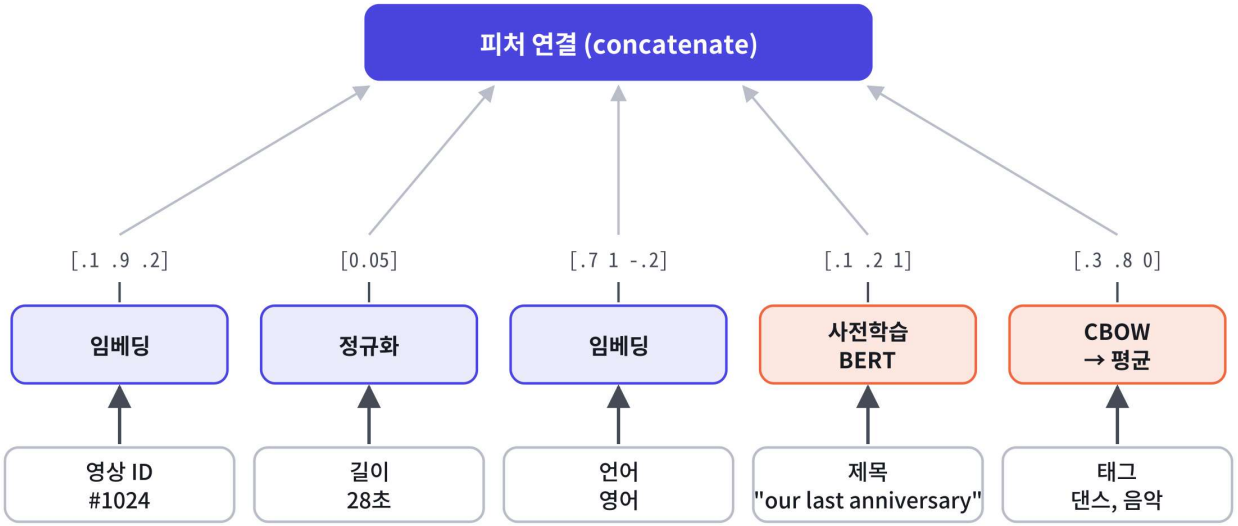


그림 6.7

피처	처리	왜 중요한가
영상 ID	임베딩 층(학습)	범주형 → 벡터
길이	수치	짧은 영상 선호 등
언어	임베딩	사용자는 특정 언어를 선호
제목	사전학습 BERT	"피자 만드는 법" → 요리
태그	CBOW → 평균	가벼운 사전학습 모델

실습에서는 제목·태그 모델 대신 태그 0/1 벡터(LAB 1과 같은 CountVectorizer) + 언어 원-핫 + 길이/60

사용자 피처 ① 인구통계 · 맥락

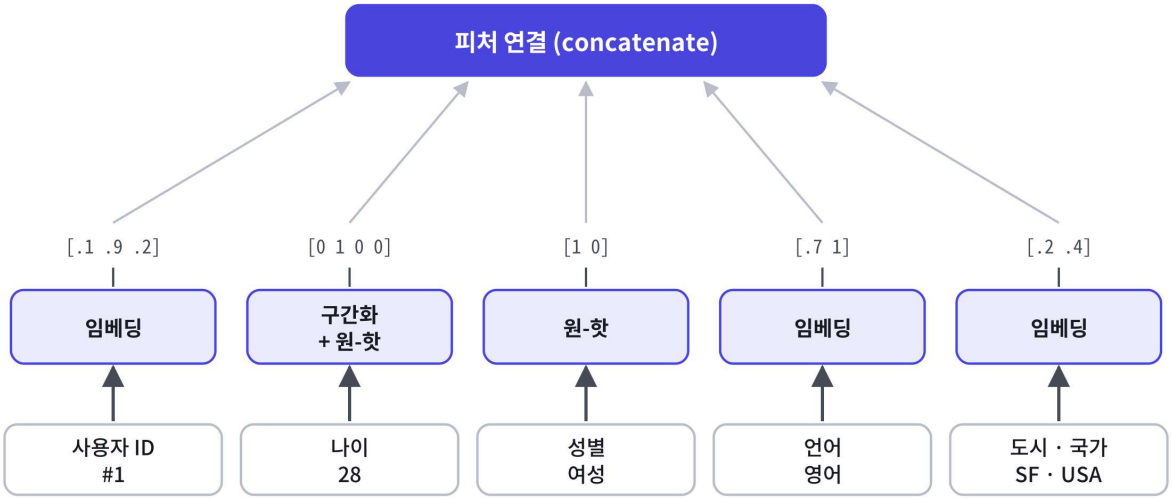


그림 6.8 — 인구통계

```
age_band = pd.cut(users.age, bins=[0, 19, 29, 39, 100])   # 구간화
demo = pd.get_dummies(...age_band, language, device...)   # 원-핫
```

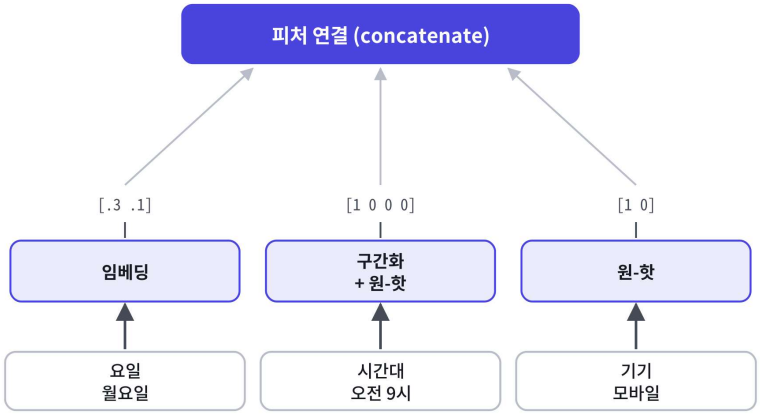
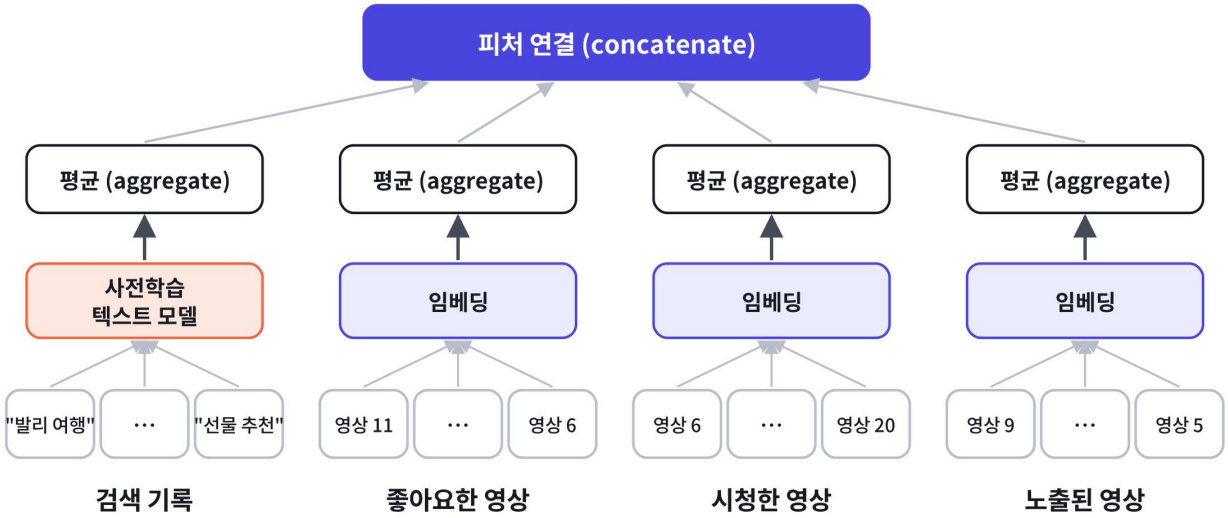


그림 6.9 — 맥락 정보

맥락 피처의 직관

- 시간대: 저녁엔 교육 영상
- 기기: 모바일은 짧은 영상
- 요일: 주말엔 다른 취향

사용자 피처 ② 과거 상호작용



개수가 사람마다 다른 목록 → 평균으로 고정 길이 벡터

그림 6.10

```
history = (A_train @ X) / n_liked      # 본 영상들의 태그 벡터 평균
```

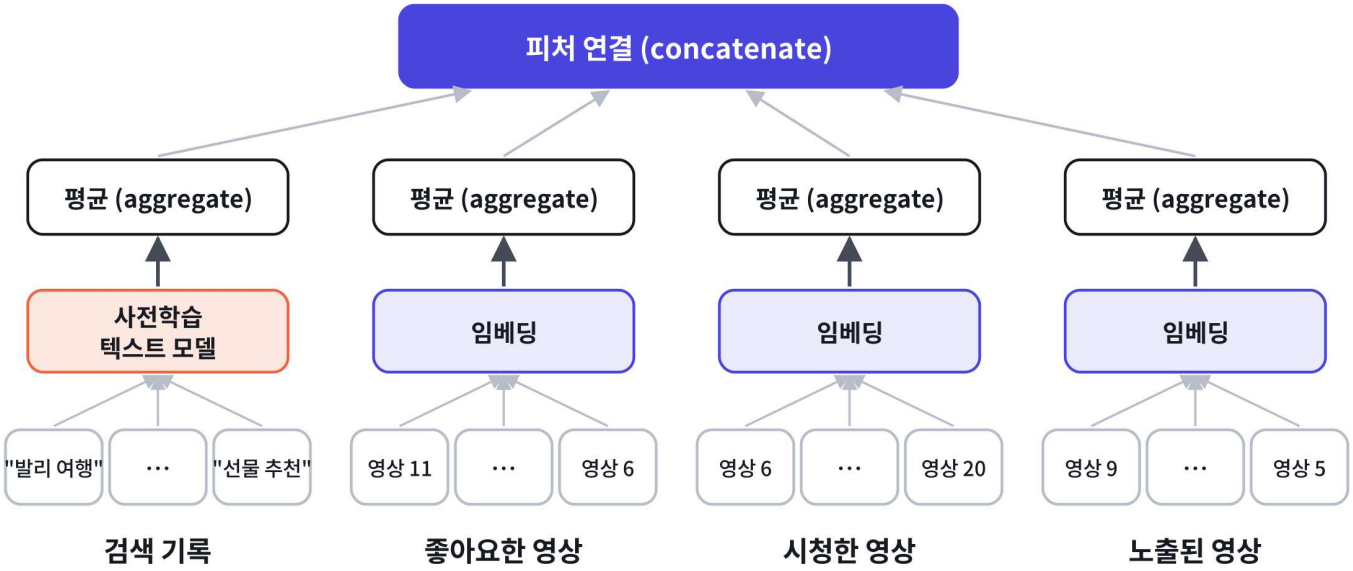
왜 중요한가

과거 행동은 미래 행동의 좋은 예측 변수다.

어떻게

- 검색어 → BERT 임베딩
- 좋아요·시청·노출 영상 → 영상 ID 임베딩
- 사람마다 개수가 다름 → 평균내어 고정 길이로

가변 길이 기록의 피처화 — 평균으로 고정 길이 벡터 만들기



개수가 사람마다 다른 목록 → 평균으로 고정 길이 벡터

그림 6.10 — 검색·좋아요·시청·노출 기록을 각각 평균내어 이어 붙인다

같은 연산을 이미 했다
실습 1의 콘텐츠 기반 프로필 = 좋아한 영상 벡터의 평균

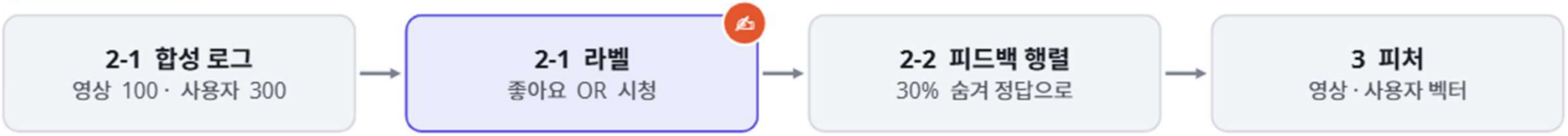
```
history = (A_train @ X) / n_liked
# 사용자 300명 모두의 평균을 행렬 곱 한 번에
```

왜 중요한가
과거 행동은 미래 행동의 좋은 예측 변수

핵심 길이가 제각각인 시청·좋아요 기록은 영상 임베딩의 평균을 내어 고정 길이 벡터로 바꿔야 모델에 입력할 수 있다

실습 2 · 관련성 라벨과 피드백 행렬 (빈칸 ③)

HANDS-ON 02 약 15 분 · 9 차시 _ 실습 _ 동영상추천시스템 .ipynb · 2~3 절



✍ 빈칸 ③

```
log['label'] = ____
```

통과 신호

통과하면 " 관련 있음 비율 = 26.1%" 가 출력됩니다 (assert 로 양성 4,691 개 확인).

힌트 : (좋아요 == 1) | (시청 ⩾ 0.5) → .astype(int)

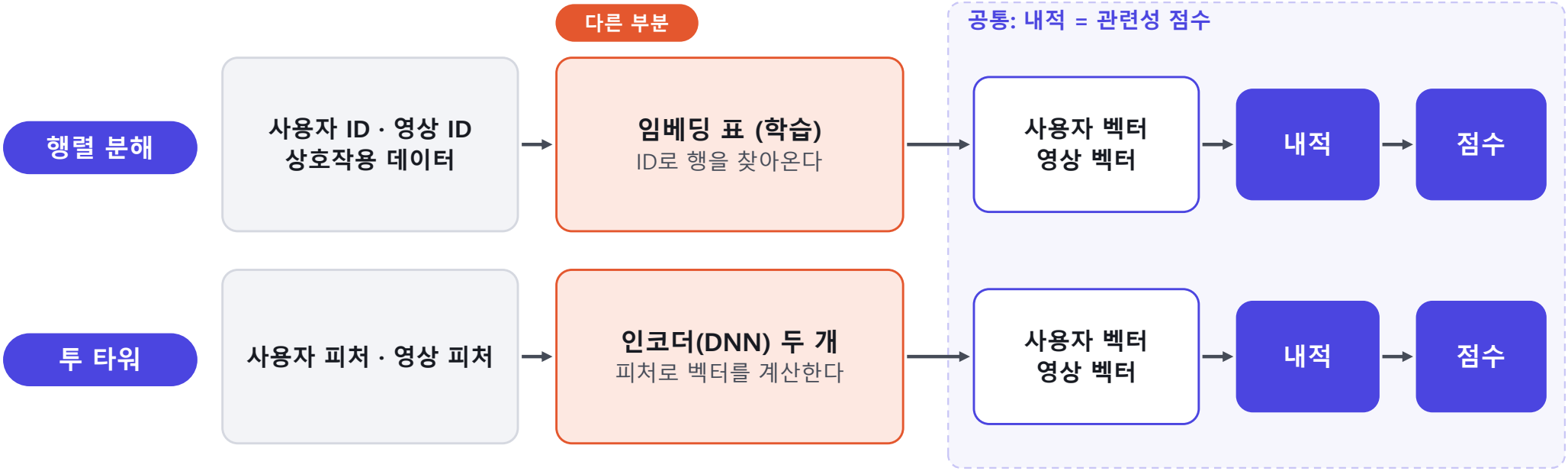
관찰 : 좋아요만 쓰면 7.9% 인데 왜 결합해야 할까요 ?

1차시 정리 — 1~3단계의 핵심 개념과 결론

	배울 핵심 개념	결론 (1차시)
① 요구사항	기능·비기능 요구사항 (규모, 지연 시간), 설계 범위	**100억 개 · 200ms → 후보를 줄이는 다단계 구조가 필요**
② 문제 정의	ML 목표 · 관련성 정의 · 콘텐츠 기반 / 협업 필터링 / 하이브리드	**목표 = 관련 영상 수 최대화 · 순차 하이브리드 채택**
③ 데이터	상호작용 로그 · 라벨 · 피드백 행렬 · 피처 엔지니어링	**라벨 = 좋아요 OR 절반 이상 시청**
④ 모델 개발	행렬 분해(손실 함수 · WALS) · 투 타워 신경망 · 모델 비교	다음 차시에서 학습
⑤ 평가	오프라인 지표 (P@k · mAP · 다양성) · 온라인 지표 (CTR 등)	다음 차시에서 학습
⑥ 서빙	후보 생성 → 점수 매기기 → 재순위 · 콜드 스타트 · 과제	다음 차시에서 학습

생각해 보기 **2차시 질문: 행렬 분해와 투 타워는 각각 어디에 쓰고, 100억 개를 200ms 안에 어떻게 줄일까?**

모델 개발 개요 — 행렬 분해와 투 타워의 공통 구조



입력도 다르고(ID vs 피처) 벡터를 얻는 방법도 다르지만, 벡터가 생긴 뒤는 완전히 같다

핵심 **행렬 분해는 ID로 임베딩 표에서 벡터를 찾고, 투 타워는 피처로 벡터를 계산한다 — 둘 다 두 벡터의 내적이 관련성 점수다**

피드백 행렬 — 무엇을 1로 칠할까



그림 6.11

실습: 300 × 100 행렬 중 15.6%만 1 — 나머지는 "모른다"

명시적 피드백

좋아요·공유. 의견을 정확히 반영하지만 행렬이 매우 희소 → 학습이 어렵다

암묵적 피드백

클릭·시청 시간. 데이터는 많지만 의견을 직접 반영하지 않고 잡음이 있다

결합 ✓ 채택

ML 목표가 "관련성 = 명시 + 암묵 결합"이므로 행렬도 같은 정의로 만든다

행렬 분해 — 임베딩의 곱으로 관련성 점수 계산

피드백 행렬 (3×5)

1	1			
		1	1	
		1		1

≈

사용자 임베딩 (3×2)

0.9	0.1
-0.7	0.1
-0.6	-0.8

×

영상 임베딩 (2×5)

0.6	0.8	-0.8	-0.6	-0.5
0	-0.2	0	0.6	-0.3

사용자 3명 × 영상 5개

행 = 차원(여기서는 2) · 열 = 영상

① 학습
U × V ≈ 피드백 행렬
(1이었던 칸은 크게, 빈칸은 작게)

→

② 추론
사용자 2 · 영상 5의 내적
(−0.7)(−0.5) + (0.1)(−0.3) = 0.32

→

③ 추천
이미 본 영상은 빼고
점수가 높은 순으로

핵심 **행렬 분해는 사용자·영상 임베딩의 내적으로 점수를 내며, 상호작용만 사용하므로 사용자·영상 피처는 쓸 수 없다**

학습 목표 — 곱이 피드백 행렬을 근사하도록



1이었던 칸은 크게, 비었던 칸은 작게 — 그렇게 되도록 임베딩을 학습한다

학습 절차

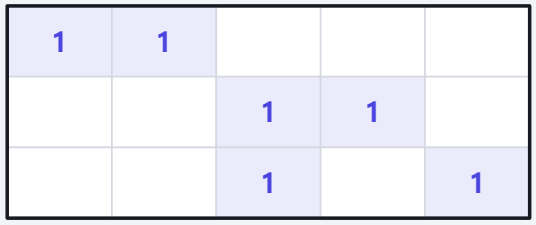
- 두 임베딩 행렬을 무작위로 초기화
- 예측 행렬과 피드백 행렬의 손실을 계산
- 손실이 줄도록 반복 갱신

그림 6.13

그런데 "손실"을 어디에서 잴까? 빈칸도 셀까? → 다음 슬라이드

행렬 분해의 손실 함수 — 곱이 피드백 행렬을 근사하도록 학습

① 관찰된 쌍만



$\Sigma_{obs}\ (A_{ij} - U_i \cdot V_j)^2$

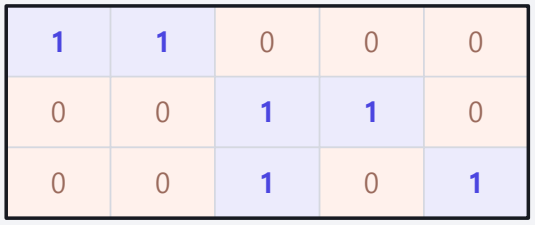
빈칸 평균 예측

0.80

P@10 **0.104**

"전부 1"이라 답해도 손실 0 — 퇴화

② 모든 쌍 똑같이



$\Sigma_{all}\ (A_{ij} - U_i \cdot V_j)^2$

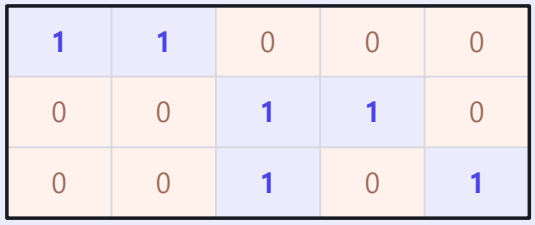
관찰칸 평균 예측

0.43

P@10 **0.220**

빈칸이 압도적으로 많아
예측이 0 쪽으로 쏠림

③ 빈칸에 작은 가중치 W (채택)



$\Sigma_{obs}(\ldots)^2 + W \cdot \Sigma_{unobs}(\ldots)^2$

관찰칸 / 빈칸 평균 예측

0.86 / 0.18

P@10 **0.225**

두 항의 균형 — 실무에서 잘 동작

생각해 보기 관찰된 칸만 학습하면 모든 칸을 1로 예측하는 모델이 된다. 빈칸에는 어느 정도의 가중치를 줘야 할까?

Feedback matrix

1	1			
		1	1	
	1		1	

Predicted scores

0.54	0.9	-0.9	-0.9	-0.45
-0.46	-0.61	0.68	0.8	0.32
-0.32	0.72	-0.16	0.8	-0.24

$$\text{loss} = \sum_{(i,j) \in obs} (A_{ij} - U_i . V_j)^2$$

Feedback matrix

1	1	0	0	0
0	0	1	1	0
0	1	0	1	0

Predicted scores

0.54	0.9	-0.9	-0.9	-0.45
-0.46	-0.61	0.68	0.8	0.32
-0.32	0.72	-0.16	0.8	-0.24

$$\text{loss} = \sum_{(i,j)} (A_{ij} - U_i . V_j)^2$$

Feedback matrix

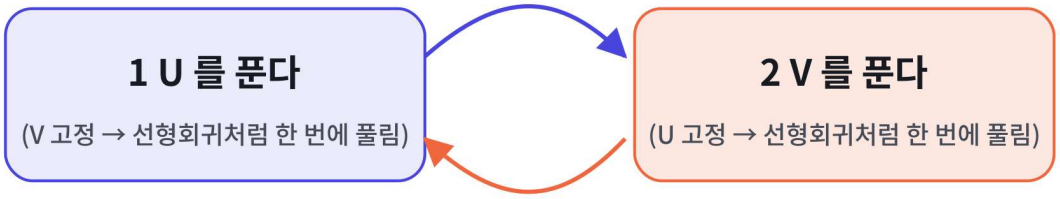
1	1	0	0	0
0	0	1	1	0
0	1	0	1	0

Predicted scores

0.54	0.9	-0.9	-0.9	-0.45
-0.46	-0.61	0.68	0.8	0.32
-0.32	0.72	-0.16	0.8	-0.24

$$\text{loss} = \sum_{(i,j) \in obs} (A_{ij} - U_i . V_j)^2 + W \sum_{(i,j) \notin obs} (A_{ij} - U_i . V_j)^2$$

행렬 분해의 학습 — WALS (교대 최소제곱)



3 번갈아 반복 — 빨리 수렴하고, 사용자·영상별로 병렬 계산이 가능하다

```
for _ in range(n_iter):
    for i in 사용자:  U[i] = solve(Vᵀ·Wᵢ·V + λI, Vᵀ·Wᵢ·Aᵢ)   # V 고정
    for j in 영상:    V[j] = solve(Uᵀ·Wⱼ·U + λI, Uᵀ·Wⱼ·Aⱼ)   # U 고정
```

최적화 방법

SGD
일반적인 경사하강법
손실을 조금씩 줄인다

WALS ✓ 채택
행렬 분해 전용
빨리 수렴 · 병렬 계산 쉬움

실습 결과
P@10 0.225 (인기순 0.083)

핵심 **WALS는 사용자 임베딩과 영상 임베딩을 번갈아 고정하며 나머지를 가중 최소제곱으로 풀어, SGD보다 빠르게 수렴한다**

행렬 인수분해 추론 — 임의의 사용자와 후보 동영상 간의 관련성은 내적으로 계산한다.

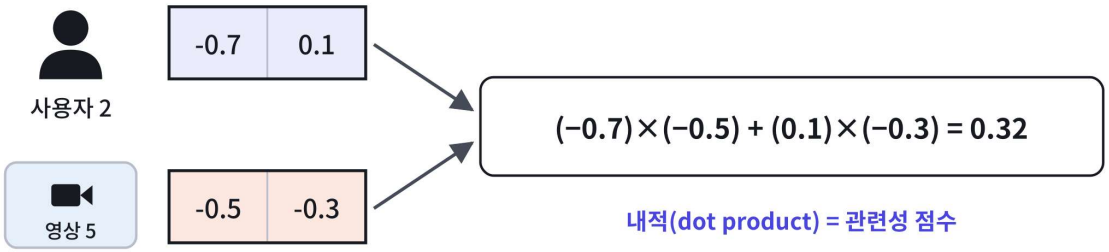


그림 6.17

```
score = U_toy[1] @ V_toy[4]  # 사용자2·영상5
# (-0.7)×(-0.5) + (0.1)×(-0.3) = 0.32

P = U_toy @ V_toy.T          # 모든 쌍
```

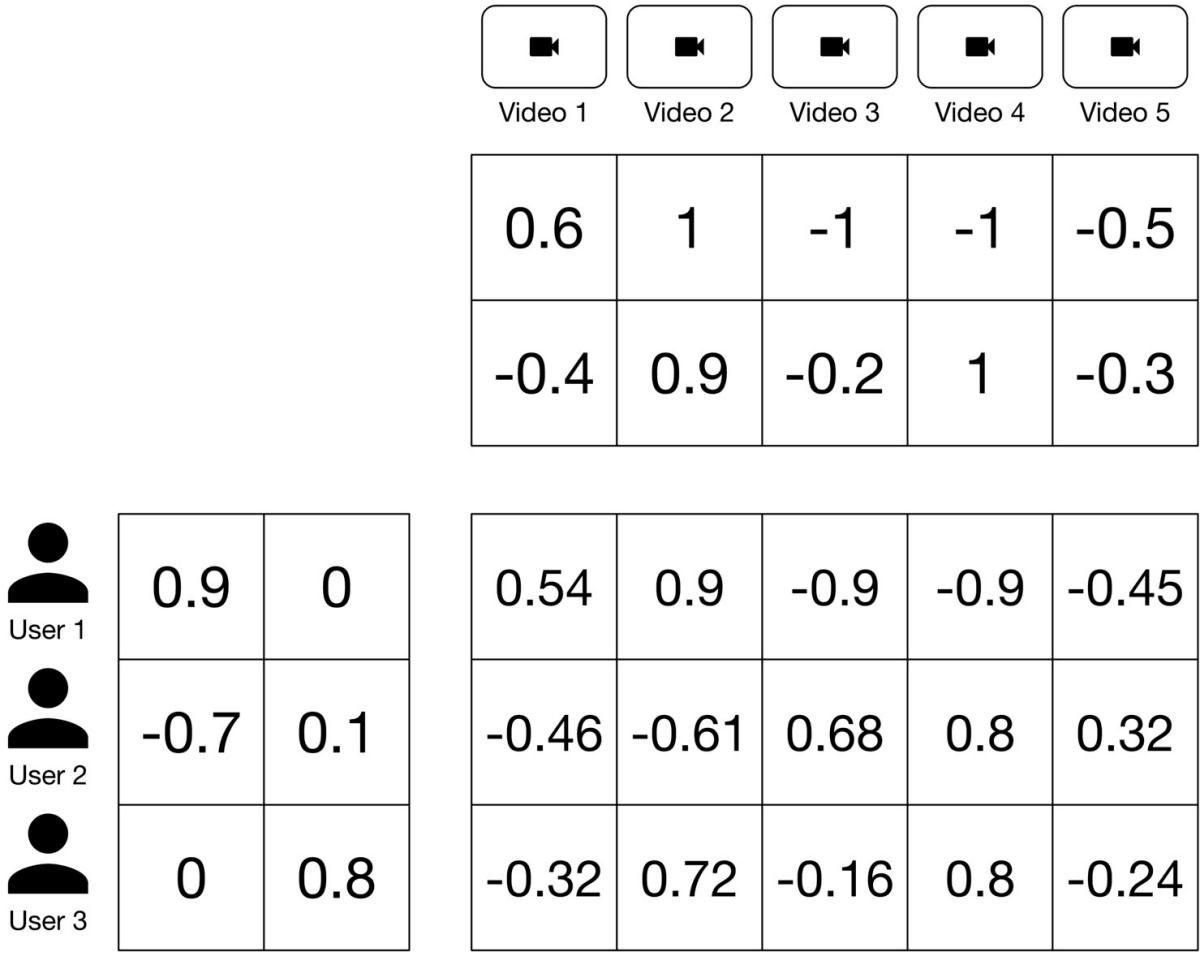
	영상1	영상2	영상3	영상4	영상5
사용자1	**0.54**	**0.70**	-0.72	-0.48	-0.48
사용자2	-0.42	-0.58	**0.56**	**0.48**	**0.32**
사용자3	-0.36	-0.32	**0.48**	-0.12	**0.54**

그림 6.18 — 모든 쌍의 예측된 연관성 점수 파란 글자 = 원래 1이던 칸

사용자 2에게: 이미 본 영상 3·4를 빼면 영상 5(0.32)가 1위

기억하기

행렬 분해는 상호작용만 쓰므로 주로 협업 필터링에 쓰인다



Feedback matrix

1	1	0	0	0
0	0	1	1	0
0	1	0	1	0

행렬 분해의 장단점

장점

- 학습이 빠르다 — 임베딩 행렬 두 개만 학습하면 됨.
- 서빙이 빠르다 — 학습된 임베딩은 고정이라 요청 때 변환이 필요 없다

단점

- 상호작용만 쓴다 — 나이·언어 같은 사용자와 동영상의 유용한 피처를 사용하지 않는다.
- 신규 사용자를 처리하기에 어렵다. — 상호작용이 없으면 의미 있는 임베딩이 없다

두 단점을 해결하려면 → 피처를 입력으로 받는 모델 = 투 타워 신경망